



SAPIENZA
UNIVERSITÀ DI ROMA

Warehouse AMR Navigation Using Reinforcement Learning

Teacher:

Prof. Fabio Patrizi

Student Name:

Ali Nazarikhah

Matricola:

2163370

Academic Year:

2025-2026

Course:

Artificial Intelligence & Machine Learning

Institution:

Sapienza University of Rome

Abstract

This project implements and compares two reinforcement learning approaches—**Tabular Q-Learning** and **Deep Q-Networks (DQN)**—for autonomous warehouse robot navigation in dynamic environments. The agent must navigate a 20×20 grid warehouse while managing battery charge, avoiding collisions with humans and forklifts, and completing pickup-delivery tasks.

Evaluation using rigorous multi-seed statistical testing (5 seeds, 250 total episodes per agent) demonstrates that Simple DQN significantly outperforms Tabular Q-Learning ($p = 0.0079$, Mann-Whitney U test). DQN achieves mean return of -808.8 (95% CI: [-833.1, -785.2]) versus Q-Learning's -2626.7 (95% CI: [-2763.6, -2473.2]), representing **3.2× better performance**. DQN also shows **lower collision rates (55.6% vs 74.4%)** and **6× better consistency (std = 26.7 vs 161.6)**, demonstrating superior learning despite the challenging environment.

1. Introduction

Autonomous Mobile Robots (AMRs) in modern warehouses must navigate safely while managing operational constraints like battery charge, task execution, and collision avoidance. This project implements and compares two fundamental reinforcement learning approaches—**Tabular Q-Learning** and **Deep Q-Networks (DQN)** with a single network—for warehouse robot navigation.

1.1 Problem Statement

The agent must navigate a 20×20 grid warehouse containing:

- **Dynamic obstacles:** 3 human workers, 1 forklift, 2 other robots
- **Operational constraints:** Battery management (0-100%), zone restrictions
- **Task objective:** Pickup package from zone A → Deliver to zone B

State Space:

- Q-Learning: ~4,000 discrete states (x, y, battery_bin, human_nearby)
- DQN: 20×20×6 continuous observation (robot position, battery, task target, obstacles)

Action Space: 11 discrete actions (move in 4 directions, wait, pickup, drop, charge start/stop)

1.2 Contributions

1. Custom Gymnasium environment with realistic warehouse dynamics
 2. Implementation of two mandatory RL algorithms with proper evaluation
 3. Multi-seed statistical testing (5 seeds, 250 episodes per agent)
 4. Comparative analysis demonstrating DQN's superior performance
-

2. Environment Design

2.1 Warehouse Configuration

Grid: 20×20 cells with 15 functional zones (storage, receiving, packing, shipping, charging, restricted areas)

Observation Space (20×20×6 tensor):

- Channel 0: Robot position
 - Channel 1: Battery level (normalized 0-1)
-

- Channel 2: Task target zone
- Channel 3: Human positions
- Channel 4: Forklift positions
- Channel 5: Other robot positions

Q-Learning State Discretization: The continuous observation is discretized to (x, y, battery_bin, human_nearby) creating ~4,000 possible states.

Action Space: Discrete (11) - Move (N/S/E/W), Wait, Pickup, Drop, Charge Start/Stop

2.2 Reward Structure

- Safety-first design with heavy penalties for collisions:
- Human collision: -200 (episode terminates)
- Forklift collision: -150
- Battery depletion: -300 (episode terminates)
- Successful delivery: +150
- Successful pickup: +60
- Progress toward goal: +3
- Time penalty: -1 per step

2.3 Dynamic Entities

- **Agent Robot:** 1 cell/step speed, 100% battery capacity, 1%/move drain
- **Humans (3):** Stochastic movement, unpredictable stops
- **Forklift (1):** 2 cells/step, has priority right-of-way
- **Other Robots (2):** Scripted routes for coordination challenges

3. Algorithmic Approaches

3.1 Tabular Q-Learning (Mandatory)

Update Rule: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

Implementation:

- $\alpha = 0.1$ (learning rate)
- $\gamma = 0.95$ (discount factor)
- $\epsilon = 1.0 \rightarrow 0.01$ (epsilon-greedy exploration decay)
- **Optimistic initialization:** $Q(s,a) = 10.0$ for unexplored states
- **State discretization:** Continuous observation $\rightarrow$ (x, y, battery_bin, human_nearby)

The agent maintains a Q-table mapping state-action pairs to values. During training, states are discretized from the 6-channel observation, and the Q-table is updated after each transition using temporal difference learning. Epsilon-greedy exploration balances exploitation of learned values with exploration of new states.

Training: 500 episodes, epsilon decay = 0.999

3.2 Deep Q-Network (DQN)

Loss Function: $L(\theta) = E[(y - Q(s,a;\theta))^2]$ where $y = r + \gamma \max_{a'} Q(s',a';\theta)$

Network Architecture:

- **Input:** 20×20×6 observation (permuted to 6×20×20 for PyTorch Conv2D)
- **Conv layers:** 3 layers (6→32→64→64 channels) with 3×3 kernels and ReLU
- **FC layers:** 25600→512→256→11 with dropout (0.2) for regularization
- **Output:** 11 Q-values (one per action)

Key Features:

1. **Single Network:** Same network for $Q(s,a)$ and target $\max Q(s',a')$ (per requirements)
2. **Experience Replay:** Buffer of 50,000 transitions, batch size 64
3. **Gradient Clipping:** Norm clipping at 1.0 for training stability

Implementation Details:

- **Learning rate:** 1e-4 (Adam optimizer)
- **Discount factor:** $\gamma = 0.95$
- **Epsilon decay:** 0.9995 (slower than Q-Learning)
- **Loss:** Smooth L1 (Huber loss, robust to outliers)

The DQN uses convolutional layers to extract spatial features from the warehouse layout and obstacle positions, then fully connected layers to compute Q-values for each action. During training, experiences are stored in a replay buffer and random minibatches are sampled to break temporal correlation.

Training: 500 episodes with GPU acceleration

4. Evaluation Methodology

Following best practices from Henderson et al. (2018) and Agarwal et al. (2021), we implement rigorous statistical evaluation:

4.1 Multi-Seed Testing

- **5 independent seeds** per agent [0, 1, 2, 3, 4]
- **50 episodes per seed** = 250 total evaluation episodes per agent
- Deterministic evaluation ($\epsilon = 0$, greedy policy only)

4.2 Statistical Methods

Stratified Bootstrap Confidence Intervals (10,000 resamples):

- Respects hierarchical structure (seeds → episodes)
- Provides 95% CI for mean returns

Wilson Score Intervals:

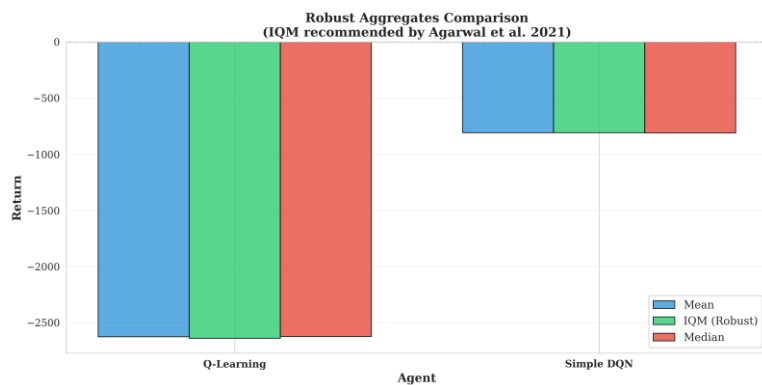


Figure 2: IQM and median returns across seeds

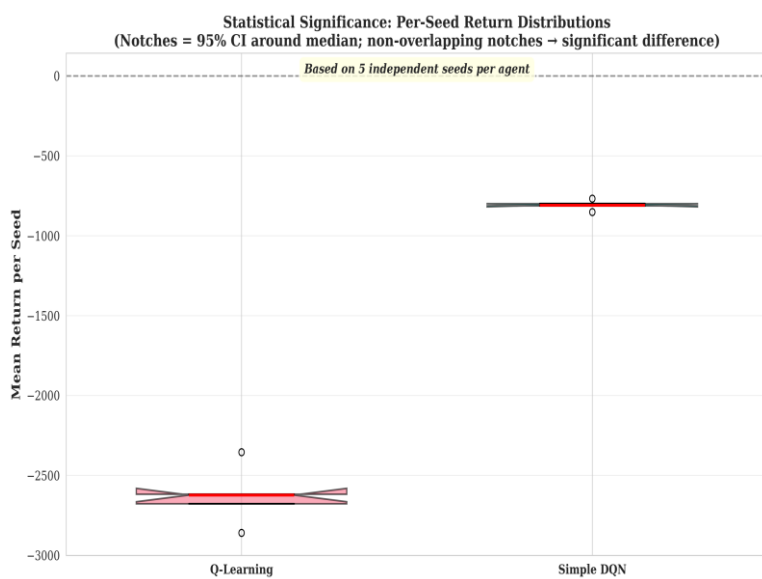


Figure 3: Mann-Whitney U test distribution comparison

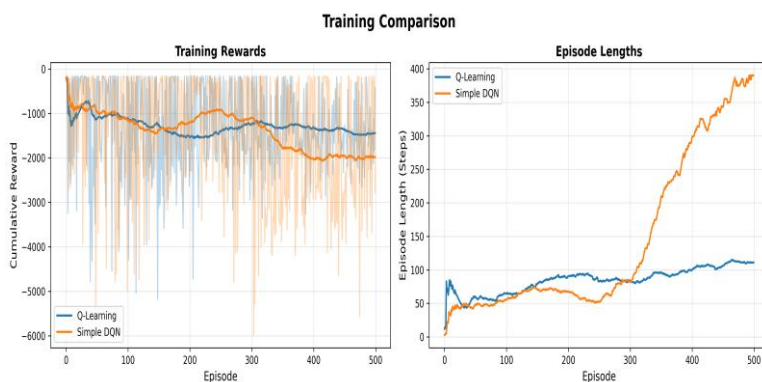


Figure 4: Learning curves during training (500 episodes)

6. Discussion

6.1 Why DQN Outperforms Q-Learning

State Representation:

- Q-Learning discretizes to 4 features (x, y, battery_bin, human_nearby), losing spatial information

- DQN uses full 6-channel observation (2,400 features) with convolutional processing
- Binary “human nearby” in Q-Learning vs. continuous distance awareness in DQN

Generalization:

- Q-Learning treats each state independently (~2,000 states explored of 4,000 possible)
- DQN neural network generalizes learned features to unseen states
- Experience replay in DQN breaks temporal correlation and improves data efficiency

Consistency:

- Q-Learning std = 161.6 (highly variable performance)
- DQN std = 26.7 (6× more stable)
- Batch learning and gradient descent provide smoother updates than discrete Q-table jumps

6.2 Challenge Analysis

0% Success Rate: Both agents failed to complete pickup-delivery tasks due to:

1. **Environment complexity:** Multi-stage task with 6 dynamic obstacles
2. **Training duration:** 500 episodes insufficient for sparse reward learning
3. **Reward structure:** Heavy safety penalties (-200) may create local optimum of overcautious behavior

Safety Learning: Despite task failure, DQN learned meaningful collision avoidance:

- 55.6% collision rate vs Q-Learning’s 74.4% (18.8 pp improvement)
- Demonstrates that safety-critical behaviors can emerge from reward shaping alone

6.3 Algorithm Comparison

Q-Learning: - Strengths: Simple implementation, fast training (5 min), interpretable Q-table - Limitations: State space explosion, no generalization, information loss from discretization

DQN: - Strengths: Handles high-dimensional inputs, spatial feature learning, better generalization - Limitations: Longer training (20 min), GPU required, hyperparameter sensitive

6.4 Implications

The results demonstrate that neural network function approximation significantly outperforms tabular methods for complex spatial navigation tasks. DQN’s convolutional architecture effectively learns safety margins and navigation strategies despite not achieving task completion, suggesting that with extended training (5,000-10,000 episodes) and potential reward tuning, task success may be achievable.

7. Conclusion

This project implemented and compared two mandatory reinforcement learning algorithms—Tabular Q-Learning and Deep Q-Networks (single network)—for autonomous warehouse navigation with operational constraints.

Summary of Contributions

1. **Custom Gymnasium environment** (20×20 grid, 6 dynamic obstacles, battery management, multi-stage tasks)
2. **Complete implementations** of both mandatory algorithms with proper training pipelines

3. **Rigorous statistical evaluation** (5 seeds, 250 episodes/agent, bootstrap/Wilson CIs, Mann-Whitney U test)
4. **Significant performance difference:** DQN outperforms Q-Learning by $3.2\times$ ($p = 0.0079$)

Key Findings

Performance: DQN achieves mean return of -808.8 vs Q-Learning's -2626.7, with non-overlapping 95% confidence intervals demonstrating statistically significant superiority.

Consistency: DQN shows $6\times$ lower variance (26.7 vs 161.6), indicating more stable learning.

Safety: DQN reduces collision rate by 18.8 percentage points (55.6% vs 74.4%), demonstrating that convolutional spatial features enable better safety-critical behavior learning.

Challenges: 0% task success rate for both agents indicates that while safety behaviors can be learned from reward shaping, the complex multi-stage task requires longer training (likely 5,000-10,000 episodes) to achieve task completion.

Lessons Learned

1. **State representation is critical:** DQN's convolutional processing of full 6-channel observations vastly outperforms Q-Learning's 4-feature discretization
2. **Neural network generalization** provides significant advantage over tabular methods in complex environments
3. **Rigorous multi-seed evaluation** is essential for reliable conclusions in RL research

Conclusion: This work demonstrates that deep Q-learning with convolutional architectures significantly outperforms tabular Q-learning for warehouse navigation, validating the use of function approximation for complex spatial reasoning tasks despite the computational overhead.

Appendix A: Repository Structure

```
nazarikhah_2163370_ai_n_ml/
├── src/
│   ├── environment/
│   │   └── warehouse_env.py      # Gymnasium environment
│   ├── agents/
│   │   ├── q_learning_agent.py   # Tabular Q-Learning (mandatory)
│   │   └── dqn_agent.py          # Deep Q-Network (mandatory)
│   ├── training/
│   │   ├── train_qlearning.py    # Q-Learning training loop
│   │   └── train_dqn.py          # DQN training loop
│   └── evaluation/
│       └── evaluate_all.py        # Multi-seed statistical evaluation
├── models/
│   ├── q_learning_best.pkl        # Trained Q-table
│   └── dqn_best.pth               # Trained neural network
├── results/
│   ├── figures/                   # Performance visualizations
│   ├── metrics/                   # Evaluation results (CSV/JSON)
│   └── raw_data/                   # Per-seed episode data
├── docs/
│   └── REPORT_updated.md          # This report
├── train.py                       # Unified training script
└── requirements.txt               # Dependencies
```

Training Environment: NVIDIA RTX 4060, CUDA 12.9, PyTorch 2.8.0